

3.7 过程

过程是软件中一种很重要的抽象。它提供了一种封装代码的方式，用一组指定的参数和一个可选的返回值实现了某种功能。然后，可以在程序中不同的地方调用这个函数。设计良好的软件用过程作为抽象机制，隐藏某个行为的具体实现，同时又提供清晰简洁的接口定义，说明要计算的是哪些值，过程会对程序状态产生什么样的影响。不同编程语言中，过程的形式多样：函数(function)、方法(method)、子例程(subroutine)、处理函数(handler)等等，但是它们有一些共有的特性。

要提供对过程的机器级支持，必须要处理许多不同的属性。为了讨论方便，假设过程 P 调用过程 Q，Q 执行后返回到 P。这些动作包括下面一个或多个机制：

传递控制。在进入过程 Q 的时候，程序计数器必须被设置为 Q 的代码的起始地址，然后在返回时，要把程序计数器设置为 P 中调用 Q 后面那条指令的地址。

传递数据。P 必须能够向 Q 提供一个或多个参数，Q 必须能够向 P 返回一个值。

分配和释放内存。在开始时，Q 可能需要为局部变量分配空间，而在返回前，又必须释放这些存储空间。

x86-64 的过程实现包括一组特殊的指令和一些对机器资源(例如寄存器和程序内存)使用的约定规则。人们花了大量的力气来尽量减少过程调用的开销。所以，它遵循了被认为是最低要求策略的方法，只实现上述机制中每个过程所必需的那些。接下来，我们一步步地构建起不同的机制，先描述控制，再描述数据传递，最后是内存管理。

3.7.1 运行时栈

C 语言过程调用机制的一个关键特性(大多数其他语言也是如此)在于使用了栈数据结构提供的后进先出的内存管理原则。在过程 P 调用过程 Q 的例子中，可以看到当 Q 在执行时，P 以及所有在向上追溯到 P 的调用链中的过程，都是暂时被挂起的。当 Q 运行时，它只需要为局部变量分配新的存储空间，或者设置到另一个过程的调用。另一方面，当 Q 返回时，任何它所分配的局部存储空间都可以被释放。因此，程序可以用栈来管理它的过程所需要的存储空间，栈和程序寄存器存放着传递控制和数据、分配内存所需要的信息。当 P 调用 Q 时，控制和数据信息添加到栈尾。当 P 返回时，这些信息会释放掉。

如 3.4.4 节中讲过的，x86-64 的栈向低地址方向增长，而栈指针 `%rsp` 指向栈顶元素。可以用 `pushq` 和 `popq` 指令将数据存入栈中或是从栈中取出。将栈指针减小一个适当的量可以为没有指定初始值的数据在栈上分配空间。类似地，可以通过增加栈指针来释放空间。

当 x86-64 过程需要的存储空间超出寄存器能够存放的大小时，就会在栈上分配空间。这个部分称为过程的栈帧(stack fram)。图 3-25

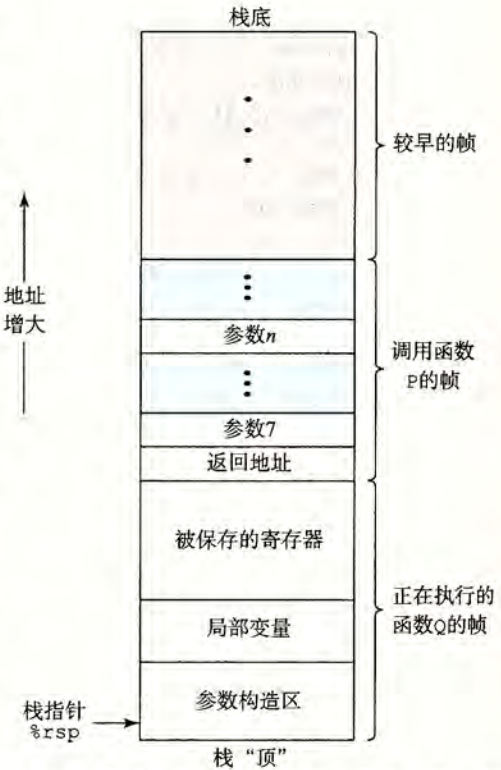


图 3-25 通用的栈帧结构(栈用来传递参数、存储返回信息、保存寄存器，以及局部存储。省略了不必要的部分)

给出了运行时栈的通用结构，包括把它划分为栈帧。当前正在执行的过程的帧总是在栈顶。当过程 P 调用过程 Q 时，会把返回地址压入栈中，指明当 Q 返回时，要从 P 程序的哪个位置继续执行。我们把这个返回地址当做 P 的栈帧的一部分，因为它存放的是与 P 相关的状态。Q 的代码会扩展当前栈的边界，分配它的栈帧所需的空間。在这个空间中，它可以保存寄存器的值，分配局部变量空间，为它调用的过程设置参数。大多数过程的栈帧都是定长的，在过程的开始就分配好了。但是有些过程需要变长的帧，这个问题会在 3.10.5 节中讨论。通过寄存器，过程 P 可以传递最多 6 个整数值(也就是指针和整数)，但是如果 Q 需要更多的参数，P 可以在调用 Q 之前在自己的栈帧里存储好这些参数。

为了提高空间和时间效率，x86-64 过程只分配自己所需要的栈帧部分。例如，许多过程有 6 个或者更少的参数，那么所有的参数都可以通过寄存器传递。因此，图 3-25 中画出的某些栈帧部分可以省略。实际上，许多函数甚至根本不需要栈帧。当所有的局部变量都可以保存在寄存器中，而且该函数不会调用任何其他函数(有时称之为叶子过程，此时把过程调用看做树结构)时，就可以这样处理。例如，到目前为止我们仔细审视过的所有函数都不需要栈帧。

3.7.2 转移控制

将控制从函数 P 转移到函数 Q 只需要简单地把程序计数器(PC)设置为 Q 的代码的起始位置。不过，当稍后从 Q 返回的时候，处理器必须记录好它需要继续 P 的执行的代码位置。在 x86-64 机器中，这个信息是用指令 `call Q` 调用过程 Q 来记录的。该指令会把地址 A 压入栈中，并将 PC 设置为 Q 的起始地址。压入的地址 A 被称为返回地址，是紧跟在 `call` 指令后面那条指令的地址。对应的指令 `ret` 会从栈中弹出地址 A，并把 PC 设置为 A。

下表给出的是 `call` 和 `ret` 指令的一般形式：

指令	描述
<code>call Label</code>	过程调用
<code>call *Operand</code>	过程调用
<code>ret</code>	从过程调用中返回

(这些指令在程序 OBJDUMP 产生的反汇编输出中被称为 `callq` 和 `retq`。添加的后缀‘q’只是为了强调这些是 x86-64 版本的调用和返回，而不是 IA32 的。在 x86-64 汇编代码中，这两种版本可以互换。)

`call` 指令有一个目标，即指明被调用过程起始的指令地址。同跳转一样，调用可以是直接的，也可以是间接的。在汇编代码中，直接调用的目标是一个标号，而间接调用的目标是 * 后面跟一个操作数指示符，使用的是图 3-3 中描述的格式之一。

图 3-26 说明了 3.2.2 节中介绍的 `multstore` 和 `main` 函数的 `call` 和 `ret` 指令的执行情况。下面是这两个函数的反汇编代码的节选：

```
Beginning of function multstore
1  000000000400540 <multstore>:
2      400540: 53                push    %rbx
3      400541: 48 89 d3          mov     %rdx,%rbx
    . . .
Return from function multstore
4      40054d: c3                retq
```



```
Call to multstore from main
5    400563: e8 d8 ff ff ff      callq 400540 <multstore>
6    400568: 48 8b 54 24 08      mov 0x8(%rsp),%rdx
```

在这段代码中我们可以看到，在 main 函数中，地址为 0x400563 的 call 指令调用函数 multstore。此时的状态如图 3-26a 所示，指明了栈指针 %rsp 和程序计数器 %rip 的值。call 的效果是将返回地址 0x400568 压入栈中，并跳到函数 multstore 的第一条指令，地址为 0x400540(图 3-26b)。函数 multstore 继续执行，直到遇到地址 0x40054d 处的 ret 指令。这条指令从栈中弹出值 0x400568，然后跳转到这个地址，就在 call 指令之后，继续 main 函数的执行。

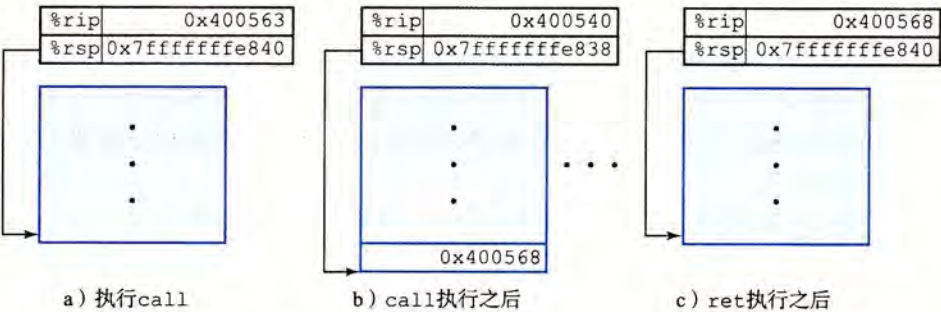


图 3-26 call 和 ret 函数的说明。call 指令将控制转移到一个函数的起始，而 ret 指令返回到这次调用后面的那条指令

再来看一个更详细说明在过程间传递控制的例子，图 3-27a 给出了两个函数 top 和 leaf 的反汇编代码，以及 main 函数中调用 top 处的代码。每条指令都以标号标出：L1~L2 (leaf 中)，T1~T4(main 中)和 M1~M2(main 中)。该图的 b 部分给出了这段代码执

```
Disassembly of leaf(long y)
y in %rdi
1  0000000000400540 <leaf>:
2    400540: 48 8d 47 02      lea 0x2(%rdi),%rax L1: y+2
3    400544: c3              retq L2: Return

4  0000000000400545 <top>:
Disassembly of top(long x)
x in %rdi
5    400545: 48 83 ef 05      sub $0x5,%rdi T1: x-5
6    400549: e8 f2 ff ff ff   callq 400540 <leaf> T2: Call leaf(x-5)
7    40054e: 48 01 c0         add %rax,%rax T3: Double result
8    400551: c3              retq T4: Return

...

Call to top from function main
9    40055b: e8 e5 ff ff ff   callq 400545 <top> M1: Call top(100)
10   400560: 48 89 c2         mov %rax,%rdx M2: Resume
```

a) 说明过程调用和返回的反汇编代码

图 3-27 包含过程调用和返回的程序的执行细节。使用栈来存储返回地址使得能够返回到过程中正确的位置

指令			状态值 (指令执行前)				描述
标号	PC	指令	%rdi	%rax	%rsp	*%rsp	
M1	0x40055b	callq	100	—	0x7fffffff820	—	调用top(100)
T1	0x400555	sub	100	—	0x7fffffff818	0x400560	进入top
T2	0x400559	callq	95	—	0x7fffffff818	0x400560	调用leaf(95)
L1	0x400540	lea	95	—	0x7fffffff810	0x40054e	进入leaf
L2	0x400544	retq	—	97	0x7fffffff810	0x40054e	从leaf返回97
T3	0x40054e	add	—	97	0x7fffffff818	0x400560	继续top
T4	0x400551	retq	—	194	0x7fffffff818	0x400560	从top返回194
M2	0x400560	mov	—	194	0x7fffffff820	—	继续main

b) 示例代码的执行过程

图 3-27 (续)

行的详细过程, main 调用 top(100), 然后 top 调用 leaf(95)。函数 leaf 向 top 返回 97, 然后 top 向 main 返回 194。前面三列描述了被执行的指令, 包括指令标号、地址和指令类型。后面四列给出了在该指令执行程序的状态, 包括寄存器 %rdi、%rax 和 %rsp 的内容, 以及位于栈顶的值。仔细研究这张表的内容, 它们说明了运行时栈在管理支持过程调用和返回所需的存储空间中的重要作用。

leaf 的指令 L1 将 %rax 设置为 97, 也就是要返回的值。然后指令 L2 返回, 它从栈中弹出 0x400054e。通过将 PC 设置为这个弹出的值, 控制转移回 top 的 T3 指令。程序成功完成对 leaf 的调用, 返回到 top。

指令 T3 将 %rax 设置为 194, 也就是要从 top 返回的值。然后指令 T4 返回, 它从栈中弹出 0x4000560, 因此将 PC 设置为 main 的 M2 指令。程序成功完成对 top 的调用, 返回到 main。可以看到, 此时栈指针也恢复成了 0x7fffffff820, 即调用 top 之前的值。

可以看到, 这种把返回地址压入栈的简单的机制能够让函数在稍后返回到程序中正确的点。C 语言(以及大多数程序语言)标准的调用/返回机制刚好与栈提供的后进先出的内存管理方法吻合。

 **练习题 3.32** 下面列出的是两个函数 first 和 last 的反汇编代码, 以及 main 函数调用 first 的代码:

Disassembly of last(long u, long v)

u in %rdi, v in %rsi

1 0000000000400540 <last>:

2	400540:	48 89 f8	mov	%rdi,%rax	L1: u
3	400543:	48 0f af c6	imul	%rsi,%rax	L2: u*v
4	400547:	c3	retq		L3: Return

Disassembly of first(long x)

x in %rdi

5 0000000000400548 <first>:

6	400548:	48 8d 77 01	lea	0x1(%rdi),%rsi	F1: x+1
7	40054c:	48 83 ef 01	sub	\$0x1,%rdi	F2: x-1
8	400550:	e8 eb ff ff ff	callq	400540 <last>	F3: Call last(x-1,x+1)


```
9      400555:  f3 c3                      repz retq          F4: Return
      :
      :
10     400560:  e8 e3 ff ff ff          callq 400548 <first> M1: Call first(10)
11     400565:  48 89 c2                mov    %rax,%rdx    M2: Resume
```

每条指令都有一个标号，类似于图 3-27a。从 main 调用 first(10) 开始，到程序返回 main 时为止，填写下表记录指令执行的过程。

指令			状态值(指令执行前)					
标号	PC	指令	%rdi	%rsi	%rax	%rsp	* %rsp	描述
M1	0x400560	callq	10	—	—	0x7fffffff820	—	调用 first(10)
F1								
F2								
F3								
L1								
L2								
L3								
F4								
M2								

3.7.3 数据传送

当调用一个过程时，除了要把控制传递给它并在过程返回时再传递回来之外，过程调用还可能包括把数据作为参数传递，而从过程返回还有可能包括返回一个值。x86-64 中，大部分过程间的数据传送是通过寄存器实现的。例如，我们已经看到无数的函数示例，参数在寄存器 %rdi、%rsi 和其他寄存器中传递。当过程 P 调用过程 Q 时，P 的代码必须首先把参数复制到适当的寄存器中。类似地，当 Q 返回到 P 时，P 的代码可以访问寄存器 %rax 中的返回值。在本节中，我们更详细地探讨这些规则。

x86-64 中，可以通过寄存器最多传递 6 个整型(例如整数和指针)参数。寄存器的使用是有特殊顺序的，寄存器使用的名字取决于要传递的数据类型的大小，如图 3-28 所示。会根据参数在参数列表中的顺序为它们分配寄存器。可以通过 64 位寄存器适当的部分访问小于 64 位的参数。例如，如果第一个参数是 32 位的，那么可以用 %edi 来访问它。

操作数大小(位)	参数数量					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

图 3-28 传递函数参数的寄存器。寄存器是按照特殊顺序来使用的，而使用的名字是根据参数的大小来确定的

如果一个函数有大于 6 个整型参数，超出 6 个的部分就要通过栈来传递。假设过程 P 调用过程 Q，有 n 个整型参数，且 n > 6。那么 P 的代码分配的栈帧必须要能容纳 7 到 n 号参数的存储空间，如图 3-25 所示。要把参数 1~6 复制到对应的寄存器，把参数 7~n 放

到栈上，而参数 7 位于栈顶。通过栈传递参数时，所有的数据大小都向 8 的倍数对齐。参数到位以后，程序就可以执行 `call` 指令将控制转移到过程 `Q` 了。过程 `Q` 可以通过寄存器访问参数，有必要的話也可以通过栈访问。相应地，如果 `Q` 也调用了某个有超过 6 个参数的函数，它也需要在自己的栈帧中为超出 6 个部分的参数分配空间，如图 3-25 中标号为“参数构造区”的区域所示。

作为参数传递的示例，考虑图 3-29a 所示的 C 函数 `proc`。这个函数有 8 个参数，包括字节数不同的整数(8、4、2 和 1)和不同类型的指针，每个都是 8 字节的。

```
void proc(long a1, long *a1p,
          int a2, int *a2p,
          short a3, short *a3p,
          char a4, char *a4p)
{
    *a1p += a1;
    *a2p += a2;
    *a3p += a3;
    *a4p += a4;
}
```

a) C代码

```
void proc(a1, a1p, a2, a2p, a3, a3p, a4, a4p)
Arguments passed as follows:
    a1 in %rdi          (64 bits)
    a1p in %rsi         (64 bits)
    a2 in %edx          (32 bits)
    a2p in %rcx         (64 bits)
    a3 in %r8w          (16 bits)
    a3p in %r9          (64 bits)
    a4 at %rsp+8        ( 8 bits)
    a4p at %rsp+16      (64 bits)

1  proc:
2  movq    16(%rsp), %rax    Fetch a4p (64 bits)
3  addq    %rdi, (%rsi)     *a1p += a1 (64 bits)
4  addl    %edx, (%rcx)     *a2p += a2 (32 bits)
5  addw    %r8w, (%r9)      *a3p += a3 (16 bits)
6  movl    8(%rsp), %edx    Fetch a4 ( 8 bits)
7  addb    %dl, (%rax)      *a4p += a4 ( 8 bits)
8  ret                                Return
```

b) 生成的汇编代码

图 3-29 有多个不同类型参数的函数示例。参数 1~6 通过寄存器传递，而参数 7~8 通过栈传递

图 3-29b 中给出 `proc` 生成的汇编代码。前面 6 个参数通过寄存器传递，后面 2 个通过栈传递，就像图 3-30 中画出来的那样。可以看到，作为过程调用的一部分，返回地址被压入栈中。因而这两个参数位于相对于栈指针距离为 8 和 16 的位置。在这段代码中，我们可以看到根据操作数的大小，使用了 `ADD` 指令的不同版本：`a1(long)` 使用 `addq`，`a2(int)` 使用 `addl`，`a3(short)` 使用 `addw`，而 `a4(char)` 使用 `addb`。请注意第 6 行的 `movl` 指令从内存读入 4 字节，而后面的 `addb` 指令只使用其中的低位一字节。

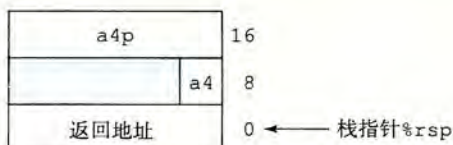


图 3-30 函数 proc 的栈帧结构。参数 a4 和 a4p 通过栈传递

练习题 3.33 C 函数 `procprob` 有 4 个参数 `u`、`a`、`v` 和 `b`，每个参数要么是一个有符号数，要么是一个指向有符号数的指针，这里的数大小不同。该函数的函数体如下：

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

编译得到如下 x86-64 代码：

```
1  procprob:
2      movslq %edi, %rdi
3      addq   %rdi, (%rdx)
4      addb   %sil, (%rcx)
5      movl   $6, %eax
6      ret
```

确定 4 个参数的合法顺序和类型。有两种正确答案。

3.7.4 栈上的局部存储

到目前为止我们看到的大多数过程示例都不需要超出寄存器大小的本地存储区域。不过有些时候，局部数据必须存放在内存中，常见的情况包括：

- 寄存器不足够存放所有的本地数据。
- 对一个局部变量使用地址运算符‘&’，因此必须能够为它产生一个地址。
- 某些局部变量是数组或结构，因此必须能够通过数组或结构引用被访问到。在描述数组和结构分配时，我们会讨论这个问题。

一般来说，过程通过减小栈指针在栈上分配空间。分配的结果作为栈帧的一部分，标号为“局部变量”，如图 3-25 所示。

来看一个处理地址运算符的例子，图 3-31a 中给出的两个函数。函数 `swap_add` 交换指针 `xp` 和 `yp` 指向的两个值，并返回这两个值的和。函数 `caller` 创建到局部变量 `arg1` 和 `arg2` 的指针，把它们传递给 `swap_add`。图 3-31b 展示了 `caller` 是如何用栈帧来实现这些局部变量的。`caller` 的代码开始的时候把栈指针减掉了 16；实际上这就是在栈上分配了 16 个字节。`S` 表示栈指针的值，可以看到这段代码计算 `&arg2` 为 `S+8`（第 5 行），而 `&arg1` 为 `S`。因此可以推断局部变量 `arg1` 和 `arg2` 存放在栈帧中相对于栈指针偏移量为 0 和 8 的地方。当对 `swap_add` 的调用完成后，`caller` 的代码会从栈上取出这两个值（第 8~9 行），计算它们的差，再乘以 `swap_add` 在寄存器 `%rax` 中返回的值（第 10 行）。最后，该函数把栈指针加 16，释放栈帧（第 11 行）。通过这个例子可以看到，运行时栈提供了一种简单的、在需要时分配、函数完成时释放局部存储的机制。

如图 3-32 所示，函数 `call_proc` 是一个更复杂的例子，说明 x86-64 栈行为的一些特性。尽管这个例子有点儿长，但还是值得仔细研究。它给出了一个必须在栈上分配局部变量存储空间的函数，同时还要向有 8 个参数的函数 `proc` 传递值（图 3-29）。该函数创建一个栈帧，如图 3-33 所示。

```

long swap_add(long *xp, long *yp)
{
    long x = *xp;
    long y = *yp;
    *xp = y;
    *yp = x;
    return x + y;
}

long caller()
{
    long arg1 = 534;
    long arg2 = 1057;
    long sum = swap_add(&arg1, &arg2);
    long diff = arg1 - arg2;
    return sum * diff;
}

```

a) swap_add和调用函数的代码

```

long caller()
1 caller:
2     subq    $16, %rsp           Allocate 16 bytes for stack frame
3     movq    $534, (%rsp)       Store 534 in arg1
4     movq    $1057, 8(%rsp)     Store 1057 in arg2
5     leaq    8(%rsp), %rsi      Compute &arg2 as second argument
6     movq    %rsp, %rdi         Compute &arg1 as first argument
7     call    swap_add           Call swap_add(&arg1, &arg2)
8     movq    (%rsp), %rdx       Get arg1
9     subq    8(%rsp), %rdx      Compute diff = arg1 - arg2
10    imulq   %rdx, %rax          Compute sum * diff
11    addq    $16, %rsp          Deallocate stack frame
12    ret                                Return

```

b) 调用函数生成的汇编代码

图 3-31 过程定义和调用的示例。由于会使用地址运算符，所以调用代码必须分配一个栈帧

```

long call_proc()
{
    long x1 = 1; int x2 = 2;
    short x3 = 3; char x4 = 4;
    proc(x1, &x1, x2, &x2, x3, &x3, x4, &x4);
    return (x1+x2)*(x3-x4);
}

```

a) swap_add和调用函数的代码

图 3-32 调用在图 3-29 中定义的函数 proc 的代码示例。该代码创建了一个栈帧


```
long call_proc()
1  call_proc:
    Set up arguments to proc
2      subq    $32, %rsp           Allocate 32-byte stack frame
3      movq    $1, 24(%rsp)       Store 1 in &x1
4      movl    $2, 20(%rsp)       Store 2 in &x2
5      movw    $3, 18(%rsp)       Store 3 in &x3
6      movb    $4, 17(%rsp)       Store 4 in &x4
7      leaq    17(%rsp), %rax      Create &x4
8      movq    %rax, 8(%rsp)       Store &x4 as argument 8
9      movl    $4, (%rsp)         Store 4 as argument 7
10     leaq    18(%rsp), %r9       Pass &x3 as argument 6
11     movl    $3, %r8d           Pass 3 as argument 5
12     leaq    20(%rsp), %rcx      Pass &x2 as argument 4
13     movl    $2, %edx           Pass 2 as argument 3
14     leaq    24(%rsp), %rsi      Pass &x1 as argument 2
15     movl    $1, %edi           Pass 1 as argument 1
    Call proc
16     call    proc
    Retrieve changes to memory
17     movslq   20(%rsp), %rdx      Get x2 and convert to long
18     addq     24(%rsp), %rdx      Compute x1+x2
19     movswl   18(%rsp), %eax      Get x3 and convert to int
20     movsbl   17(%rsp), %ecx      Get x4 and convert to int
21     subl     %ecx, %eax          Compute x3-x4
22     cltq                                          Convert to long
23     imulq    %rdx, %rax          Compute (x1+x2) * (x3-x4)
24     addq     $32, %rsp          Deallocate stack frame
25     ret                                           Return
```

b) 调用函数生成的汇编代码

图 3-32 (续)

看看 call_proc 的汇编代码(图 3-32b)，可以看到代码中一大部分(第 2~15 行)是为调用 proc 做准备。其中包括为局部变量和函数参数建立栈帧，将函数参数加载至寄存器。如图 3-33 所示，在栈上分配局部变量 x1~x4，它们具有不同的大小：24~31(x1)，20~23(x2)，18~19(x3)和 17(s3)。用 leaq 指令生成到这些位置的指针(第 7、10、12 和 14 行)。参数 7(值为 4)和 8(指向 x4 的位置的指针)存放在栈中相对于栈指针偏移量为 0 和 8 的地方。

当调用过程 proc 时，程序会开始执行图 3-29b 中的代码。如图 3-30 所示，参数 7 和 8 现在位于相对于栈指针偏移量为 8 和 16 的地方，因为返回地址这时已经被压入栈中了。

当程序返回 call_proc 时，代码会取出 4 个局部变量(第 17~20 行)，并执行最终的计算。在程序结束前，把栈指针加 32，释放这个栈帧。

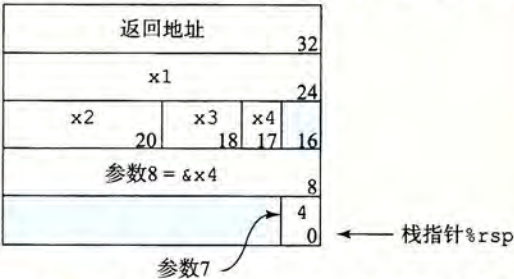


图 3-33 函数 call_proc 的栈帧。该栈帧包含局部变量和两个要传递给函数 proc 的参数

3.7.5 寄存器中的局部存储空间

寄存器组是唯一被所有过程共享的资源。

虽然在给定时刻只有一个过程是活动的，我们仍然必须确保当一个过程(调用者)调用另一个过程(被调用者)时，被调用者不会覆盖调用者稍后会使用的寄存器值。为此，x86-64 采用了一组统一的寄存器使用惯例，所有的过程(包括程序库)都必须遵循。

根据惯例，寄存器%rbx、%rbp 和 %r12~%r15 被划分为被调用者保存寄存器。当过程 P 调用过程 Q 时，Q 必须保存这些寄存器的值，保证它们的值在 Q 返回到 P 时与 Q 被调用时是一样的。过程 Q 保存一个寄存器的值不变，要么就是根本不去改变它，要么就是把原始值压入栈中，改变寄存器的值，然后在返回前从栈中弹出旧值。压入寄存器的值会在栈帧中创建标号为“保存的寄存器”的一部分，如图 3-25 中所示。有了这条惯例，P 的代码就能安全地把值存在被调用者保存寄存器中(当然，要先把之前的值保存到栈上)，调用 Q，然后继续使用寄存器中的值，不用担心值被破坏。

所有其他的寄存器，除了栈指针%rsp，都分类为调用者保存寄存器。这就意味着任何函数都能修改它们。可以这样来理解“调用者保存”这个名字：过程 P 在某个此类寄存器中有局部数据，然后调用过程 Q。因为 Q 可以随意修改这个寄存器，所以在调用之前首先保存好这个数据是 P(调用者)的责任。

来看一个例子，图 3-34a 中的函数 P。它两次调用 Q。在第一次调用中，必须保存 x 的值以备后面使用。类似地，在第二次调用中，也必须保存 Q(y) 的值。图 3-34b 中，可以看到 GCC 生成的代码使用了两个被调用者保存寄存器：%rbp 保存 x 和 %rbx 保存计算出来的

```
long P(long x, long y)
{
    long u = Q(y);
    long v = Q(x);
    return u + v;
}
```

a) 调用函数

```
long P(long x, long y)
x in %rdi, y in %rsi
1  P:
2      pushq    %rbp                Save %rbp
3      pushq    %rbx                Save %rbx
4      subq     $8, %rsp            Align stack frame
5      movq     %rdi, %rbp          Save x
6      movq     %rsi, %rdi          Move y to first argument
7      call     Q                   Call Q(y)
8      movq     %rax, %rbx          Save result
9      movq     %rbp, %rdi          Move x to first argument
10     call     Q                   Call Q(x)
11     addq     %rbx, %rax           Add saved Q(y) to Q(x)
12     addq     $8, %rsp            Deallocate last part of stack
13     popq     %rbx                Restore %rbx
14     popq     %rbp                Restore %rbp
15     ret
```

b) 调用函数生成的汇编代码

图 3-34 展示被调用者保存寄存器使用的代码。在第一次调用中，必须保存 x 的值，第二次调用中，必须保存 Q(y) 的值

$Q(y)$ 的值。在函数的开头，把这两个寄存器的值保存到栈中(第 2~3 行)。在第一次调用 Q 之前，把参数 x 复制到 `%rbp`(第 5 行)。在第二次调用 Q 之前，把这次调用的结果复制到 `%rbx`(第 8 行)。在函数的结尾，(第 13~14 行)，把它们从栈中弹出，恢复这两个被调用者保存寄存器的值。注意它们的弹出顺序与压入顺序相反，说明了栈的后进先出规则。

 **练习题 3.34** 一个函数 P 生成名为 $a_0 \sim a_7$ 的局部变量，然后调用函数 Q ，没有参数。

GCC 为 P 的第一部分产生如下代码：

```

long P(long x)
x in %rdi
1  P:
2      pushq    %r15
3      pushq    %r14
4      pushq    %r13
5      pushq    %r12
6      pushq    %rbp
7      pushq    %rbx
8      subq     $24, %rsp
9      movq     %rdi, %rbx
10     leaq     1(%rdi), %r15
11     leaq     2(%rdi), %r14
12     leaq     3(%rdi), %r13
13     leaq     4(%rdi), %r12
14     leaq     5(%rdi), %rbp
15     leaq     6(%rdi), %rax
16     movq     %rax, (%rsp)
17     leaq     7(%rdi), %rdx
18     movq     %rdx, 8(%rsp)
19     movl     $0, %eax
20     call    Q

```

- A. 确定哪些局部值存储在被调用者保存寄存器中。
- B. 确定哪些局部变量存储在栈上。
- C. 解释为什么不能把所有的局部值都存储在被调用者保存寄存器中。

3.7.6 递归过程

前面已经描述的寄存器和栈的惯例使得 x86-64 过程能够递归地调用它们自身。每个过程调用在栈中都有它自己的私有空间，因此多个未完成调用的局部变量不会相互影响。此外，栈的原则很自然地就提供了适当的策略，当过程被调用时分配局部存储，当返回时释放存储。

图 3-35 给出了递归的阶乘函数的 C 代码和生成的汇编代码。可以看到汇编代码使用寄存器 `%rbx` 来保存参数 n ，先把已有的值保存在栈上(第 2 行)，随后在返回前恢复该值(第 11 行)。根据栈的使用特性和寄存器保存规则，可以保证当递归调用 $r_{\text{fact}}(n-1)$ 返回时(第 9 行)，(1)该次调用的结果会保存在寄存器 `%rax` 中，(2)参数 n 的值仍然在寄存器 `%rbx` 中。把这两个值相乘就能得到期望的结果。

从这个例子我们可以看到，递归调用一个函数本身与调用其他函数是一样的。栈规则提供了一种机制，每次函数调用都有它自己私有的状态信息(保存的返回位置和被调用者保存寄存器的值)存储空间。如果需要，它还可以提供局部变量的存储。栈分配和释放的

规则很自然地就与函数调用-返回的顺序匹配。这种实现函数调用和返回的方法甚至对更复杂的情况也适用,包括相互递归调用(例如,过程 P 调用 Q, Q 再调用 P)。

```
long rfact(long n)
{
    long result;
    if (n <= 1)
        result = 1;
    else
        result = n * rfact(n-1);
    return result;
}
```

a) C代码

```
long rfact(long n)
n in %rdi
1  rfact:
2  pushq %rbx           Save %rbx
3  movq  %rdi, %rbx     Store n in callee-saved register
4  movl  $1, %eax       Set return value = 1
5  cmpq  $1, %rdi       Compare n:1
6  jle   .L35           If <=, goto done
7  leaq  -1(%rdi), %rdi  Compute n-1
8  call  rfact          Call rfact(n-1)
9  imulq %rbx, %rax      Multiply result by n
10 .L35:               done:
11 popq  %rbx           Restore %rbx
12 ret                    Return
```

b) 生成的汇编代码

图 3-35 递归的阶乘程序的代码。标准过程处理机制足够用来实现递归函数

 **练习题 3.35** 一个具有通用结构的 C 函数如下:

```
long rfun(unsigned long x) {
    if ( _____ )
        return _____;
    unsigned long nx = _____;
    long rv = rfun(nx);
    return _____;
}
```

GCC 产生如下汇编代码:

```
long rfun(unsigned long x)
x in %rdi
1  rfun:
2  pushq %rbx
3  movq  %rdi, %rbx
4  movl  $0, %eax
5  testq %rdi, %rdi
```



```
6    je      .L2
7    shrq    $2, %rdi
8    call    rfun
9    addq    %rbx, %rax
10   .L2:
11   popq    %rbx
12   ret
```

- A. rfun 存储在被调用者保存寄存器%rbx 中的值是什么？
- B. 填写上述 C 代码中缺失的表达式。

3.8 数组分配和访问

C 语言中的数组是一种将标量数据聚集成更大数据类型的方式。C 语言实现数组的方式非常简单，因此很容易翻译成机器代码。C 语言的一个不同寻常的特点是可以产生指向数组中元素的指针，并对这些指针进行运算。在机器代码中，这些指针会被翻译成地址计算。

优化编译器非常善于简化数组索引所使用的地址计算。不过这使得 C 代码和它到机器代码的翻译之间的对应关系有些难以理解。

3.8.1 基本原则

对于数据类型 T 和整型常数 N ，声明如下：
 $T\ A[N];$

起始位置表示为 x_A 。这个声明有两个效果。首先，它在内存中分配一个 $L \cdot N$ 字节的连续区域，这里 L 是数据类型 T 的大小(单位为字节)。其次，它引入了标识符 A ，可以用 A 来作为指向数组开头的指针，这个指针的值就是 x_A 。可以用 $0 \sim N-1$ 的整数索引来访问该数组元素。数组元素 i 会被存放在地址为 $x_A + L \cdot i$ 的地方。

作为示例，让我们来看看下面这样的声明：

```
char    A[12];
char    *B[8];
int      C[6];
double  *D[5];
```

这些声明会产生带下列参数的数组：

数组	元素大小	总的大小	起始地址	元素 i
A	1	12	x_A	$x_A + i$
B	8	64	x_B	$x_B + 8i$
C	4	24	x_C	$x_C + 4i$
D	8	40	x_D	$x_D + 8i$

数组 A 由 12 个单字节(char)元素组成。数组 C 由 6 个整数组成，每个需要 8 个字节。B 和 D 都是指针数组，因此每个数组元素都是 8 个字节。

x86-64 的内存引用指令可以用来简化数组访问。例如，假设 E 是一个 int 型的数组，而我们想计算 $E[i]$ ，在此，E 的地址存放在寄存器%rdx 中，而 i 存放在寄存器%rcx 中。然后，指令

```
movl (%rdx,%rcx,4),%eax
```

会执行地址计算 $x_E + 4i$ ，读这个内存位置的值，并将结果存放到寄存器%eax 中。允许的